Advanced Computer Systems 2026

Device Driver Project: Weeks 4-7 (worth 29%)

Due: Project Demonstrated in Week 7. Writeup due at end of week 8.. See the Assessment section of this document for more detail.

Submission: Code must be submitted under the submission link on the course website. You must also demonstrate your code and be able to answer questions about your code.

Mode: Individual work. (No group work or group write up)

Introduction

Device drivers are an important mechanism to interface between the operating system and the hardware. The hardware will appear as a file in the operating system in the /dev/directory. The Operating System defines a set of methods that must be used in the device driver. Each of these methods link to the file operation such as open(), read(), write(), seek() , etc.

In this assignment you will be developing a device driver (kernel module) to drive an 8-digit seven segment display as shown in Figure 1. Communication between the RPi and the display will be via mode 0 based SPI. You will be required to write a kernel module to implement the specified functionality on the display. You will generate the SPI protocol in software rather than in hardware. This will require you to connect GPIO pins on the RPi to the input pins on the display and then drive the state of the pins as per the SPI protocol.

The display's controller IC is the MAX7219 which acts upon the SPI signals and drives the segments on each of the digits. The datasheet for the Max7219 will be supplied on the course website.

In the N79_4.09 laboratory you will use an RPi4.

Figure 1: SPI 8-Digit 7-Segment Display showing the 8 digits and their orientation.

Project Description

The assignment requires the development of a device driver as well as the development of a user program that illustrates the operation of the device driver.

Suggested approach to the project: will take 2 parts:

1. Sessions 1/2. In this session you will write a small kernel module to become familiar with GPIO on the RPi and to become familiar with creating a simple kernel module. You will also write a code to generate SPI signals using GPIO functions for SPI communication. (see Appendix C)
2. Sessions 2-4. These sessions will involve you writing the kernel driver and user programs which involve writing the SPI comms and interpreting the commands.

The aim of these sessions are to write a kernel module to use the spi driven 8-digit seven segment display to act like a 8 character display. The requirements and functionality of the kernel module is given in the Kernel Module section. Two User programs will also need to be written that demonstrate the functionality of the display. The requirements of the User programs are given in the User Program section.

Kernel Module

The decimal point of the digit will be used to indicate the current cursor position. i.e the digit which corresponds to the current cursor position should have its decimal point set.

Open Perform the display test and set the cursor to the first digit.

Close Clear the display

Read Read the characters from the left hand side of the display to the current cursor position and return them to the user as a string.

Write Write the characters supplied from a string and write them to the display at from the current cursor position. Wrap around the right hand side of the display when required.

Seek Set the cursor position using all available modes.

Other required functionality:

- Set the display brightness level (Intensity).
- Reset the display by blanking and setting the cursor position to the start.
- Read the display brightness level (Intensity)
- Toggle the display of the cursor

SPI The SPI protocol must be implemented by manipulating the GPIO pins of the RPi to give the SPI signals rather than using the SPI subsystem which is not enabled.

- Select 3 Pins to implement the SPI protocol for the display
- Select the 3.3V pin and GND pin to power the display.
- Examine the 8-digit seven segment display to determine which SPI signals are required.

User Programs

Two user programs need to written.

1. The first user program should demonstrate the functionality of all parts of the kernel module.
2. The second user program is a digital clock. The time and date must alternate on the display. The seconds must be indicated by a dash (dot) flashing every second on the last display.
   An alarm can be set and the alarm will be indicated on the display by continually scrolling a message on the display until the # key is pressed

   This program must allow adjustment of the display brightness in increments or decrements of 10%.
   Increase brightness by 10% when the '>' key is pressed
   Decrease the brightness by 10% when the '<' key is pressed.

   The program should finish scrolling and terminate when the <ESC> key is pressed.

Assessment (24%)

You must submit your code at the end of each lab session via the submission point of the course website. This will archive your code for later referral if required. You may continue to work on the tasks between lab sessions
You may continue to work on the tasks after each lab session but still need to submit and demonstrate your final code at the start of the week 7 laboratory session.

Project Report (5%)

You are required to write a formal report in IEEE paper format. The report is to be no longer than 8 pages excluding appendices. The writeup should fully explain and justify your program design and demonstrate the correct operation of your program. The report must be submitted in pdf format. The report and code are to be submitted via the submission point of the course website.

The report is to be submitted by the end of Week 8 laboratory.

Appendix A: RPi4 Header Pinouts.

Note:

You can type “pinout in the terminal and this will show the position of the pins on the rpi4 board. (it will display a diagram of header)

Image from https://roboticsbackend.com/raspberry-pi-3-pins

Appendix B: Interfacing to the MAX7219 from the Raspberry Pi

Kernel Module SPI

- The MAX7219 internal registers are programmed using SPI. The SPI protocol used is as follows:

- The SPI mode used for communication is mode 0.

- Use the *udelay(unsigned long microsecs)* function to generate delays in micro seconds. The udelay() function requires the <linux/delay.h> header file.

- The MAX7219 requires the data to be sent MSbit First (Most Significant Bit).

- The CS is a low active signal. The CS must be asserted prior to transmission of the data and must be deasserted after transmission of the data. This is so that the data is latched into the MAX7219 command register as indicated by the address. (See page 7 of the MAX7219 datasheet).

MAX7219 Operation

- Refer to the datasheet as it is the absolute reference document. This document just gives a few hints to help you decode the datasheet protocol.

- The data command is 16 bits in size and the data consists of address[15:8] : data[7:0]. The data should be shifted into the MAX7219 MSbit first.

- The address information is given from Page 7 to Page 10 of the datasheet. The addresses can be thought of as a command/operation where the address contains the unique code number for that command/operation. (Actually the address refers to the register address in the memory map of the MAX7219)

- There are only 16 commands available for the MAX7219 hence the command forms the Least Significant Nibble of the address. The address code is of the form:

  address format: **0xX<Hex Digit>**

  where X is a don't care and has no meaning

  <Hex Digit> refers to command

  Note that **0x** is the C language prefix for a hexadecimal value

  . For simplicity I suggest that you set any X to 0 eg 0x0C for the Shutdown command.

- The data refers to data needed for that command.
  You need to carefully check the data format for each command because the meaning of the data can change based upon the command used.

Shutdown

- The shutdown command is used to do the following:
    - disable the display (data: 0)
    - enable the display (data : 1)

- All operations with the MAX7219 must follow the following sequence:
    1. Disable display
    2. Configure the display for what you want
    3. Enable the display

Reset / Initialisation Sequence

I recommend the following reset sequence for the initialisation of the display.

1. Disable display
2. Turn off the test mode (Just in case the display is in test mode)
3. Set intensity to 14/16 brightness (this is a nice sub maximal level of brightness)
4. Set the scan limit to 8 digits (this is the number of digits that the display will use)
5. Set to decode all digits for characters (See page 8 Table 5 in the datasheet for the characters)
6. Enable display

Appendix C: Accessing the GPIO pins using kernel functions

Required Include file

```
#include <linux/gpio/consumer.h>
```

Select the gpio pin to access

```
 struct gpio_desc *gpio_to_desc(unsigned int
gpio);
```

- gpio is the global Linux GPIO number, not BCM or header pin.
- Returns a descriptor or NULL if invalid.
- The GPIO number should be a multidigit number eg 554 for the Activity LED
- Declare a pointer to hold the gpio pin information structure (struct gpio_desc)

  Eg

  ```
  struct gpio_desc *gpio554 ;
  ```

Configuring direction and value

Direction

```
int gpiod_direction_input(struct gpio_desc *desc); int
gpiod_direction_output(struct gpio_desc *desc, int value);
```

- Set line as input or output.
- For outputs, value is 0/1 initial level.

set / get value

```
int gpiod_set_value(struct gpio_desc *desc, int value); int
gpiod_get_value(struct gpio_desc *desc);
```

- value is 0 or 1.
- gpiod_get_value() returns 0/1 or a negative errno on error.

Determining the pin

- The **global gpio pins** used in the above functions are the globally defined Debian Pins named gpio-XXX in the output from the command.

  Where XXX is the special global pin number. XXX is (512 + GPIO number).

  Eg the global gpio pin name for GPIO 2 is gpio-514

  XXX is calculated using (512+2)

- Alternatively In the terminal type sudo cat /sys/kernel/debug/gpio to get the Debian Pin List.
- The BCM header pins and the other pins should be apparent in the list.

Appendix D: Suggested approacvh before writing the display module

1. Write a simple kernel module that implements only the init and exit modules. The init module should write a message to the message log stating that the module has been inserted and the exit module should also write a message stating that it has been removed. The module can be tested by inserting and removing the kernel object (.ko). All the messages can be viewed using dmesg.

2. Modify Task 1 to use the GPIO to drive GPIO 26. The GND is located conveniently on the pin below it. (See Appendix A for a pinout of the RPi header). Set GPIO 26 pin high in the __init function and set the GPIO 26 pin low on the __exit function. Verify this waveform with an oscilloscope or a multimeter.

3. Modify task 2 so that the toggles GPIO 26 with a 500 usec delay between toggles. (1khZ 50% DUTY SIGNAL). You can use the udelay() function to generate the 500usec delay. Verify this waveform with the oscilloscope.

4. Create a set of functions in a file called spi.c that will be used for the spi communication. Configure the makefile for a multi-source file compilation. Use these functions to send a 'UUUU' message 20 times. Verify the spi communications using an oscilloscope.

5. You will now be ready to start the full device driver dvelopment.

Appendix E: Useful Linux kernel delay functions

The delay functions require the following include file.

```
#include <linux/delay.h>
```

The following functions spin in a loop until the desired time has passed:

void ndelay(unsigned long nsec) **// nanosecond delay**

– Delays for *nsec* nanoseconds.

void udelay(unsigned long usec) **// microsecond delay**

– Delays for usec microseconds;

– suitable for very short delays, typically under 1 millisecond.

Appendix F: Typical Device Driver Makefiles

Single Source file

```
obj-m += mydriver.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Multiple Source file

```
# Name of your final kernel module (MUST NOT match any .c file name)
MODULE_NAME = mydriver

# List the object files needed to compose the final module
$(MODULE_NAME)-y := main.o hardware.o utils.o

# Kbuild syntax tells the kernel build system this is a loadable module
obj-m := $(MODULE_NAME).o

# Path to the kernel headers/build directory
KDIR ?= /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

Hint: You may want to add more targets to these Makefile to insert a module, remove a module, compile a user file, or any other useful functionality you would like

Appendix G: Hints for writing the device driver

- Always check return values of functions and take appropriate action on the error.
- Document your code as much as possible
- Output messages to dmesg using the pr_err() pr_info() macros
    - The message should print the module name followed by your message
    - Eg pr_info("open:\topened successful\n") ; etc
- Since dmesg is huge, use dmesg | tail -20 to see the last 20 lines since new messages are added to the end of the dmesg output.
- If a module wont load, it is probably because it is already loaded.
- Check all external signal protocols with an oscilloscope to verify timing and signal correctness.
- Use dynamic allocation where possible. Attach the dynamically allocated address to the private_data field of the FILE structure
- Use the offset field (f_pos) of the FILE structure – do not globally create your own variable to track this.
- Globally declared variables should be minimised and declared with a static and should contain the name of the driver eg static int lcd_buffersize;

Appendix H: Log Level (reporting macros)

Priority	Macro	Description
1	pr_emerg()	Used for emergency messages, usually those that precede a crash.
2	pr_alert()	A situation requiring immediate action.
3	pr_crit()	Used to report a critical condition. Often related to serious hardware or software failures.
4	pr_err()	Used to report error condition.; Device drivers often use pr_err() to report hardware difficulties.
5	pr_warning()	Warnings about problematic situations that do not, in themselves, create serious problems with the system.
6	pr_notice()	Situations that are normal, but still worthy of note. A number of security-related conditions are reported at this level.
7	pr_info()	Informational messages. Many drivers print information about the hardware they find at startup time at this level.
8	pr_debug()	Used for debugging messages.